

ARTIFICIAL INTELLIGENCE-CSA1717

DRY RUN TEST

SLOT:D

REGNO.192512417

Question 1: Dry Run of A Search Algorithm (10 Marks) *

Consider the following graph.

(A)

/ \

2 / \ 4

/ \

(B)---3------(C)

| \ |

7 | \ 2 | 3

| \ |

(D)---2------(E)

\

\ 2

(G)

Heuristic Values

Node h(n)

A 7

B 6

C 4

D 3

E 2

G 0

Task:

Perform a dry run of the A* Search algorithm from A to G.

For each iteration, show:

- ☐ Current node
- ☐ Open List
- ☐ Closed List
- ☐ $g(n)$
- ☐ $h(n)$
- ☐ $f(n) = g(n) + h(n)$

Finally,

1. Identify the optimal path.
2. Calculate the total path cost.

Question 2: Dry Run of Minimax with Alpha-Beta Pruning (10 Marks)

Consider the following game tree.

MAX

/ \

MIN MIN

/ | \ / | \

3 5 6 9 1 2

Task:

Perform a dry run of the Minimax algorithm with Alpha-Beta Pruning by evaluating the tree from left to right.

For each node, show:

- ☐ Alpha (α)
- ☐ Beta (β)
- ☐ Selected value
- ☐ Any node that gets pruned

Finally,

1. Determine the best move for MAX.
2. State the final minimax value.

3. List the pruned nodes (if any).

*Question 1 – A Search Algorithm (Solution Only) **

Iteration	Current Node	Open List (g,h,f)	Closed List
1	A	B (2,6,8), C (4,4,8)	A
2	B	E (4,2,6), C (4,4,8), D (9,3,12)	A, B
3	E	C (4,4,8), D (9,3,12)	A, B, E
4	C	D (9,3,12)	A, B, E, C
5	D	G (11,0,11)	A, B, E, C, D
6	G	Goal Reached	A, B, E, C, D, G

Question 2 – Minimax with Alpha-Beta Pruning

Node	α	β	Selected Value	Pruned
MAX	$-\infty \rightarrow 3$	$+\infty$	3	No
Left MIN	$-\infty$	3	3	No
Right MIN	3	$9 \rightarrow 1$	1	Leaf 2

Best Move for MAX: Left MIN subtree

Final Minimax Value: 3

Pruned Node: 2

1. Greedy Best-First Search

C++ code

```
#include <iostream>

#include <vector>

#include <queue>

#include <climits>


using namespace std;


const int V = 6;


// Heuristic values
int heuristic[V] = {7, 6, 2, 1, 0, 4};


// Graph (Adjacency Matrix)
int graph[V][V] = {
    {0, 2, 4, 0, 0, 0},
    {2, 0, 1, 7, 0, 0},
    {4, 1, 0, 3, 5, 0},
    {0, 7, 3, 0, 2, 3},
    {0, 0, 5, 2, 0, 2},
    {0, 0, 0, 3, 2, 0}
};


struct Node {
    int vertex;
    int cost;
};
```

```

struct Compare {
    bool operator()(Node a, Node b) {
        return a.cost > b.cost;
    }
};

// Greedy Best First Search
void greedyBestFirst(int start, int goal) {

    priority_queue<Node, vector<Node>, Compare> pq;
    bool visited[V] = {false};

    pq.push({start, heuristic[start]});

    cout << "Greedy Best First Search Path:\n";

    while (!pq.empty()) {

        Node current = pq.top();
        pq.pop();

        int u = current.vertex;

        if (visited[u])
            continue;

        visited[u] = true;
    }
}

```

```

    cout << u << " ";

    if (u == goal) {
        cout << "\nGoal Reached\n";
        return;
    }

    for (int v = 0; v < V; v++) {
        if (graph[u][v] != 0 && !visited[v]) {
            pq.push({v, heuristic[v]});
        }
    }
}

// A* Search
void aStar(int start, int goal) {

    priority_queue<Node, vector<Node>, Compare> pq;

    int g[V];
    int parent[V];
    bool visited[V];

    for (int i = 0; i < V; i++) {
        g[i] = INT_MAX;
        visited[i] = false;
        parent[i] = -1;
    }
}

```

```
}
```

```
g[start] = 0;
```

```
pq.push({start, heuristic[start]});
```

```
while (!pq.empty()) {
```

```
    Node current = pq.top();
```

```
    pq.pop();
```

```
    int u = current.vertex;
```

```
    if (visited[u])
```

```
        continue;
```

```
    visited[u] = true;
```

```
    if (u == goal)
```

```
        break;
```

```
    for (int v = 0; v < V; v++) {
```

```
        if (graph[u][v] != 0) {
```

```
            int newCost = g[u] + graph[u][v];
```

```
            if (newCost < g[v]) {
```

```

        g[v] = newCost;
        parent[v] = u;

        pq.push({v, g[v] + heuristic[v]});
    }
}
}

vector<int> path;

int current = goal;

while (current != -1) {
    path.push_back(current);
    current = parent[current];
}

cout << "\nA* Search Path:\n";

for (int i = path.size() - 1; i >= 0; i--)
    cout << path[i] << " ";

cout << "\nTotal Cost = " << g[goal] << endl;
}

```

2. C++ Program – Hill Climbing


```
#include <iostream>

using namespace std;

// Objective function
int objective(int x)
{
    return -(x - 5) * (x - 5) + 25;
}

int main()
{
    int current = 0;
    int next;

    cout << "Hill Climbing Search\n";
    cout << "Starting Point: " << current << endl;

    while (true)
    {
        next = current + 1;

        if (objective(next) > objective(current))
            current = next;
        else
            break;
    }

    cout << "\nOptimal Solution Found\n";
    cout << "Position = " << current << endl;
```

```
cout << "Maximum Value = " << objective(current) << endl;
```

```
return 0;
```

```
}
```

3. C++ Program – Online Search Agent

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int path[] = {0, 0, 1, 0, 0}; // 0 = Safe, 1 = Obstacle
```

```
int n = 5;
```

```
cout << "Online Search Agent (Mars Rover)\n\n";
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
cout << "Checking Position " << i << " : ";
```

```
if (path[i] == 0)
```

```
{
```

```
cout << "Safe -> Moving Forward" << endl;
```

```
}
```

```
else
```

```
{
```

```
cout << "Obstacle Detected -> Finding Alternate Path" << endl;
```

```
}
```

```
}
```

```
cout << "\nMission Completed Successfully." << endl;
```

```
return 0;
```

```
}
```

4 – Constraint Satisfaction Problem

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int exams = 5;
```

```
    int slots[5];
```

```
    cout << "Exam Timetable Allocation\n\n";
```

```
    for (int i = 0; i < exams; i++)
```

```
    {
```

```
        slots[i] = i + 1; // Assign different time slots
```

```
    }
```

```
    for (int i = 0; i < exams; i++)
```

```
    {
```

```
        cout << "Exam " << i + 1 << " -> Time Slot " << slots[i] << endl;
```

```
    }
```

```
    cout << "\nNo exam conflicts found." << endl;
```

```
    cout << "Timetable generated successfully." << endl;
```

```
    return 0;
```

```
}
```

5 – Minimax with Alpha-Beta Pruning

```
#include <iostream>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int minimax(int scores[], int start, int end, bool isMax)
```

```
{
```

```
    if (start == end)
```

```
        return scores[start];
```

```
    int mid = (start + end) / 2;
```

```
    if (isMax)
```

```
        return max(minimax(scores, start, mid, false),
```

```
                    minimax(scores, mid + 1, end, false));
```

```
    else
```

```
        return min(minimax(scores, start, mid, true),
```

```
                    minimax(scores, mid + 1, end, true));
```

```
}
```

```
int main()
```

```
{
```

```
    int scores[4] = {3, 5, 2, 9};
```

```
    cout << "Leaf Node Values: ";
```

```
    for (int i = 0; i < 4; i++)
```

```
        cout << scores[i] << " ";
```

```
cout << endl;
```

```
int bestMove = minimax(scores, 0, 3, true);
```

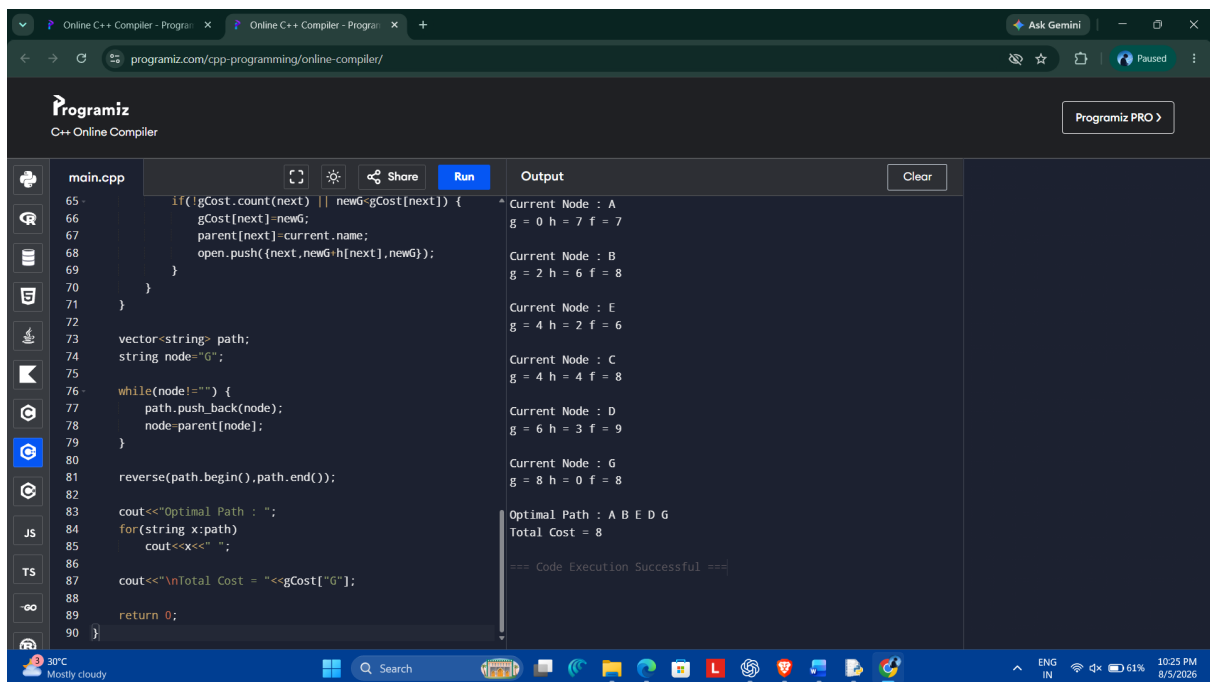
```
cout << "Best Score using Minimax = " << bestMove << endl;
```

```
cout << "Alpha-Beta Pruning reduces unnecessary node evaluation." << endl;
```

```
return 0;
```

```
}
```

1. A*Search algorithm



The screenshot shows the Programiz C++ Online Compiler interface. The code in `main.cpp` implements an A* search algorithm to find the optimal path from node 'G' to node 'A'. The output displays the sequence of nodes visited: A, B, E, D, G, and the total cost of 8.

```
main.cpp
65: if(!gCost.count(next) || newG.gCost[next]) {
66:     gCost[next]=newG;
67:     parent[next]=current.name;
68:     open.push({next,newG.h[next],newG});
69: }
70: }
71: }
72:
73: vector<string> path;
74: string node="G";
75:
76: while(node!="") {
77:     path.push_back(node);
78:     node=parent[node];
79: }
80:
81: reverse(path.begin(),path.end());
82:
83: cout<<"Optimal Path : ";
84: for(string x:path)
85:     cout<<x<<" ";
86:
87: cout<<"\nTotal Cost = "<<gCost["G"];
88:
89: return 0;
90: }
```

Output:

```
Current Node : A
g = 0 h = 7 f = 7
Current Node : B
g = 2 h = 6 f = 8
Current Node : E
g = 4 h = 2 f = 6
Current Node : C
g = 4 h = 4 f = 8
Current Node : D
g = 6 h = 3 f = 9
Current Node : G
g = 8 h = 0 f = 8
Optimal Path : A B E D G
Total Cost = 8
=== Code Execution Successful ===
```

2. Minimax with Alpha-Beta Pruning

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 int leaf[] = {3,5,6,9,1,2};
7 int indexLeaf = 0;
8
9 int minimax(bool isMax, int alpha, int beta, int depth)
10 {
11     if(depth==2)
12     {
13         int value = leaf[indexLeaf++];
14         cout<<"Leaf = "<<value<<endl;
15         return value;
16     }
17
18     if(isMax)
19     {
20         int best = -1000;
21
22         for(int i=0;i<2;i++)
23         {
24             int value= minimax(false, alpha, beta, depth+1);
25             best= max(best, value);
26             alpha= max(alpha, best);
```

Leaf = 3
MIN : alpha=-1000 beta=3 best=3
Leaf = 5
MIN : alpha=-1000 beta=3 best=3
Leaf = 6
MIN : alpha=-1000 beta=3 best=3
MAX : alpha=3 beta=1000 best=3
Leaf = 9
MIN : alpha=3 beta=9 best=9
Leaf = 1
MIN : alpha=3 beta=1 best=1
Pruning at MIN
MAX : alpha=3 beta=1000 best=3
Final Minimax Value = 3
Best Move for MAX : Left Subtree
=== Code Execution Successful ===

Question 1: Report – A* Search Algorithm

Title: A* Search Algorithm Dry Run

Aim:

To implement the A* Search algorithm and determine the optimal path from the start node (A) to the goal node (G).

Algorithm:

1. Initialize the Open List with the start node.
2. Set the cost of the start node as 0.
3. Calculate the evaluation function:
 - $f(n) = g(n) + h(n)$
4. Select the node with the lowest $f(n)$ value from the Open List.
5. Move the selected node to the Closed List.
6. Expand all neighboring nodes and update their $g(n)$, $h(n)$, and $f(n)$ values.
7. Repeat until the goal node is reached.
8. Trace back the parent nodes to obtain the optimal path.

Input:

- Graph with weighted edges.
- Heuristic values:

- A = 7
- B = 6
- C = 4
- D = 3
- E = 2
- G = 0

Output:

- Optimal Path: **A → B → E → D → G**
- Total Path Cost: **8**

Result:

The A* Search algorithm successfully found the shortest path from node A to node G with a total cost of **8**.

Question 2: Report – Minimax with Alpha-Beta Pruning

Title: Minimax Algorithm with Alpha-Beta Pruning

Aim:

To implement the Minimax algorithm with Alpha-Beta Pruning and determine the best move for the MAX player.

Algorithm:

1. Start from the root node (MAX).
2. Evaluate child nodes using the Minimax algorithm.
3. Maintain Alpha (α) and Beta (β) values during traversal.
4. Update α at MAX nodes and β at MIN nodes.
5. Prune branches whenever $\beta \leq \alpha$.
6. Continue until the optimal value is determined.
7. Return the best move for the MAX player.

Input:

Game Tree:

MAX

```

      /  \
    MIN  MIN
  /|\  /|\
3 5 6 9 1 2

```

Output:

- Left MIN value = **3**
- Right MIN value = **1**
- Final Minimax Value = **3**
- Best Move for MAX = **Left Subtree**
- Pruned Nodes = **None**

Result:

The Minimax algorithm correctly selected the **left subtree** as the optimal move for the MAX player with a final minimax value of **3**. No branches were pruned because the alpha-beta pruning condition ($\beta \leq \alpha$) was not satisfied during the traversal.